

Volume 10 – Backend Engineering & Microservices Design

AI Quant Trading Platform

Version: 1.0

Document Type: Backend Engineering Design

Classification: Core Backend Architecture

1. Purpose

This document defines the backend architecture, engineering standards, service boundaries, internal communication, project structure, coding patterns, and scalability strategy for the AI Quant Trading Platform.

The backend is designed as a modular, service-oriented system that supports continuous market analysis, automated trading, and enterprise-grade SaaS capabilities.

2. Technology Stack

Backend

- Python 3.13+
 - Django
 - Django REST Framework
 - FastAPI (high-performance services)
 - Celery
 - Redis
 - PostgreSQL
 - WebSockets
 - Docker
-

Infrastructure

- Nginx
- Unicorn
- Uvicorn
- Prometheus
- Grafana

- ELK Stack
 - GitHub Actions
-

3. Project Structure

```
backend/

├── apps/
│   ├── accounts/
│   ├── users/
│   ├── exchanges/
│   ├── market/
│   ├── scanner/
│   ├── indicators/
│   ├── ai_engine/
│   ├── ml_engine/
│   ├── strategies/
│   ├── risk/
│   ├── orders/
│   ├── portfolio/
│   ├── reports/
│   ├── notifications/
│   ├── billing/
│   ├── admin_panel/
│   └── monitoring/
├── core/
├── common/
├── config/
├── workers/
├── scheduler/
├── websocket/
├── services/
├── plugins/
├── tests/
└── docs/
```

4. Architectural Principles

The backend shall follow:

- Clean Architecture
 - Domain-Driven Design (DDD)
 - SOLID Principles
 - Repository Pattern
 - Service Layer Pattern
 - Dependency Injection
 - Event-Driven Communication
 - CQRS where beneficial
 - Stateless Services
-

5. Core Applications

Accounts

Responsibilities

- Login
 - Registration
 - Authentication
 - JWT
 - OAuth
 - Sessions
-

Users

Responsibilities

- Profiles
 - Preferences
 - Settings
 - Permissions
-

Exchanges

Responsibilities

- Exchange adapters
- API authentication
- Balance synchronization
- Order submission

- WebSocket management
 - Exchange failover
-

Market

Responsibilities

- Price ingestion
 - OHLCV storage
 - Order book updates
 - Funding rates
 - Open interest
-

Scanner

Responsibilities

- Scan markets
 - Candidate filtering
 - Signal generation
-

AI Engine

Responsibilities

- AI scoring
 - Probability estimation
 - Explainable recommendations
-

Machine Learning

Responsibilities

- Feature generation
 - Model inference
 - Retraining orchestration
 - Model registry integration
-

Strategy Engine

Responsibilities

- Entry validation

- Exit logic
 - Strategy execution
-

Risk Engine

Responsibilities

- Position sizing
 - Exposure checks
 - Drawdown limits
 - Emergency stop
-

Orders

Responsibilities

- Order lifecycle
 - Position management
 - Execution history
-

Portfolio

Responsibilities

- Balances
 - Allocation
 - PnL
 - Performance metrics
-

Reports

Responsibilities

- Analytics
 - Exports
 - Tax reports
-

Notifications

Responsibilities

- Email
- Telegram

- WhatsApp
 - Push notifications
-

Billing

Responsibilities

- Subscriptions
 - Plans
 - Invoices
 - Usage limits
-

Monitoring

Responsibilities

- Health checks
 - Metrics
 - Queue monitoring
 - Worker monitoring
-

6. Service Layer

Every module follows:

API Layer

↓

Application Service

↓

Domain Logic

↓

Repository

↓

Database

Business rules must remain outside controllers.

7. Repository Pattern

Repositories manage:

- CRUD
- Filtering
- Pagination
- Query optimization

Repositories never contain business logic.

8. Background Workers

Workers

Market Scanner

Indicator Calculator

AI Processor

ML Inference

Portfolio Sync

Exchange Sync

Notification Sender

Report Generator

Backtesting Worker

Cleanup Worker

Workers must remain independent.

9. Scheduler

Responsibilities

- Market scans
- Daily reports
- Weekly reports

- Backups
- Retraining jobs
- Cleanup tasks

Supports cron-style scheduling.

10. Event Bus

Events

UserRegistered

ExchangeConnected

ScannerCompleted

SignalGenerated

TradeExecuted

PositionClosed

RiskAlert

NotificationRequested

ReportGenerated

Events are asynchronous.

11. WebSocket Layer

Channels

Market Prices

Portfolio Updates

Order Updates

Scanner Results

AI Recommendations

Notifications

Supports automatic reconnect.

12. Plugin Architecture

Support plugins for:

- Exchanges
- Indicators
- Strategies
- Notifications
- Reports
- AI models

Plugins communicate through defined interfaces.

13. Caching Strategy

Redis caches:

- Market prices
- AI scores
- Indicator values
- User sessions
- Portfolio summaries
- Frequently used settings

Cache invalidation rules must be documented.

14. Error Handling

Centralized exception handling.

Categories

- Validation
- Authentication
- Exchange
- Business Logic
- Infrastructure
- Timeout
- Unknown

All errors receive unique codes.

15. Logging

Every service logs:

- Request ID
- User ID
- Exchange ID
- Strategy ID
- Response Time
- Errors
- Warnings

Structured JSON logging preferred.

16. Configuration Management

Configuration sources:

- Environment variables
- Secure secrets manager
- Database configuration
- Feature flags

Avoid hard-coded values.

17. Security

- JWT authentication
 - RBAC
 - Encrypted secrets
 - HTTPS
 - API rate limiting
 - Audit logs
 - Secure API credential storage
-

18. Testing Strategy

Each module requires:

- Unit tests
- Integration tests
- API tests
- Worker tests

CI pipelines must execute tests before deployment.

19. Deployment

Services run in isolated containers.

Support:

- Docker Compose
- Kubernetes (future)

Rolling deployments preferred.

20. Scalability

Independent scaling for:

- Scanner workers
- AI services
- WebSocket servers
- API servers
- Notification workers

Stateless services allow horizontal scaling.

21. Coding Standards

- Type hints required
 - Docstrings for public APIs
 - Consistent naming conventions
 - Dependency injection preferred
 - Linting and formatting enforced
 - Code review required for production
-

22. Observability

Expose metrics for:

- API latency
- Worker throughput
- Queue size
- Scanner duration
- Order execution time

- Error rates

Integrate with Prometheus and Grafana.

23. Failure Recovery

- Automatic retries
 - Circuit breakers
 - Graceful degradation
 - Queue persistence
 - Dead-letter queues
 - Health-based restarts
-

24. Design Principles

- Keep services loosely coupled.
- Favor asynchronous processing for long-running tasks.
- Separate business logic from infrastructure.
- Prefer configuration over code changes.
- Ensure every critical workflow is observable and auditable.